



Maschinelles Lernen

2. Lineare Regression

Prof. Dr. Dennis Janka

Ziele dieses Kapitels

Das erste und einfachste Modell des Supervised Learning: **Lineare Regression**

Warum?

- Trotz — bzw. *wegen* — seiner Einfachheit in Anwendungen nach wie vor relevant
- Die eingeführten Konzepte (Modellfunktion, Verlustfunktion, Training) sind **universell**
- Einstieg in die **Notation** (lineare Algebra, Vektoren, Matrizen)

Gliederung

2.1 Einfache lineare Regression

2.2 Multiple lineare Regression

2.3 Nichtlineare Features

2.4 Bias-Variance-Tradeoff





Einfache lineare Regression



Der Advertising-Datensatz

Sie sind Data-Science-Consultant

Aufgabe: Umsatz eines Produkts durch
Anpassung des Werbebudgets erhöhen

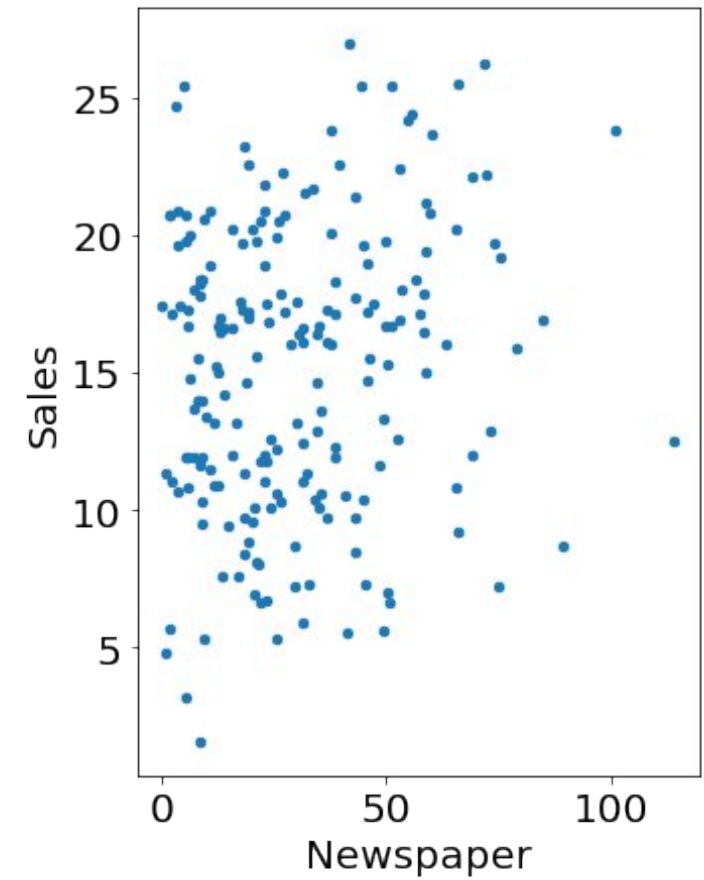
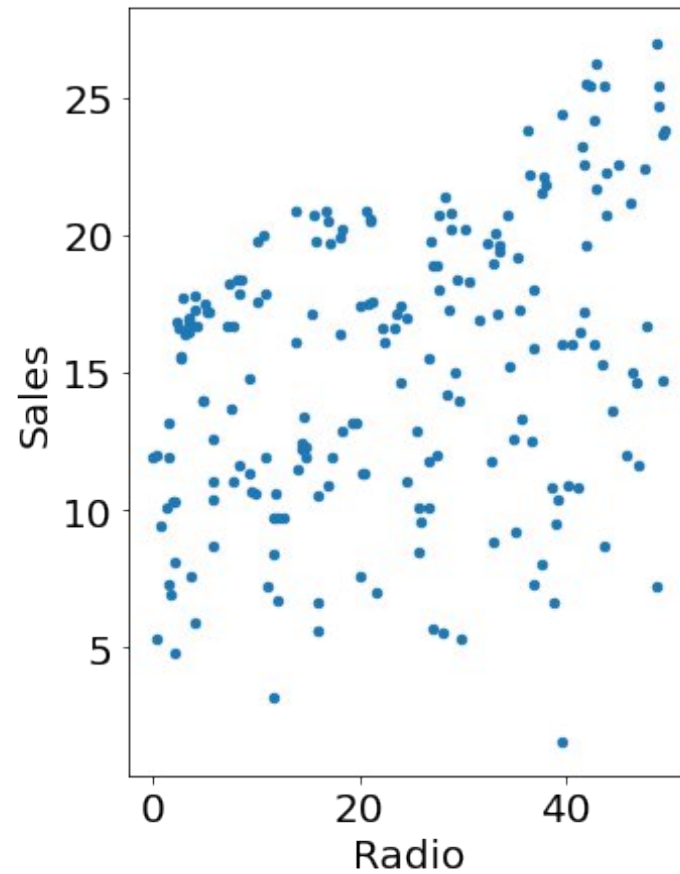
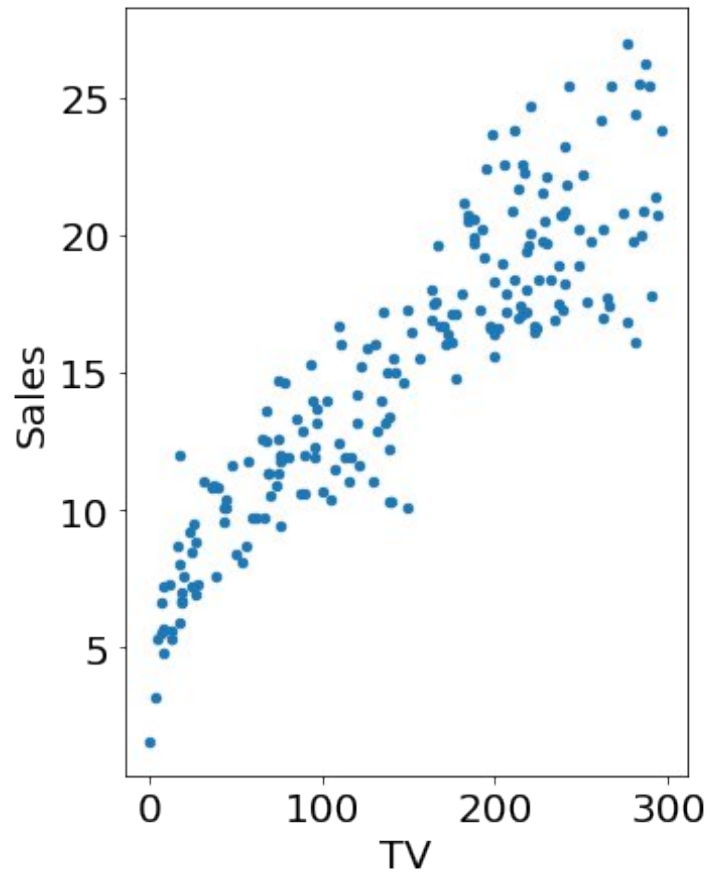
Verfügbar: Aufzeichnungen aus **200 Märkten**

- TV-Werbebudget [T€]
- Radio-Werbebudget [T€]
- Zeitungs-Werbebudget [T€]
- Umsatz [M€]

Frage: Gibt es einen Zusammenhang zwischen
Werbebudget und Umsatz?

	TV [T€]	Radio [T€]	Newspaper [T€]	Sales [M€]
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9
...	...	...	...	...
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	14.0
197	177.0	9.3	6.4	14.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	18.4

Streudiagramme



Werbebudgets vs. Umsatz

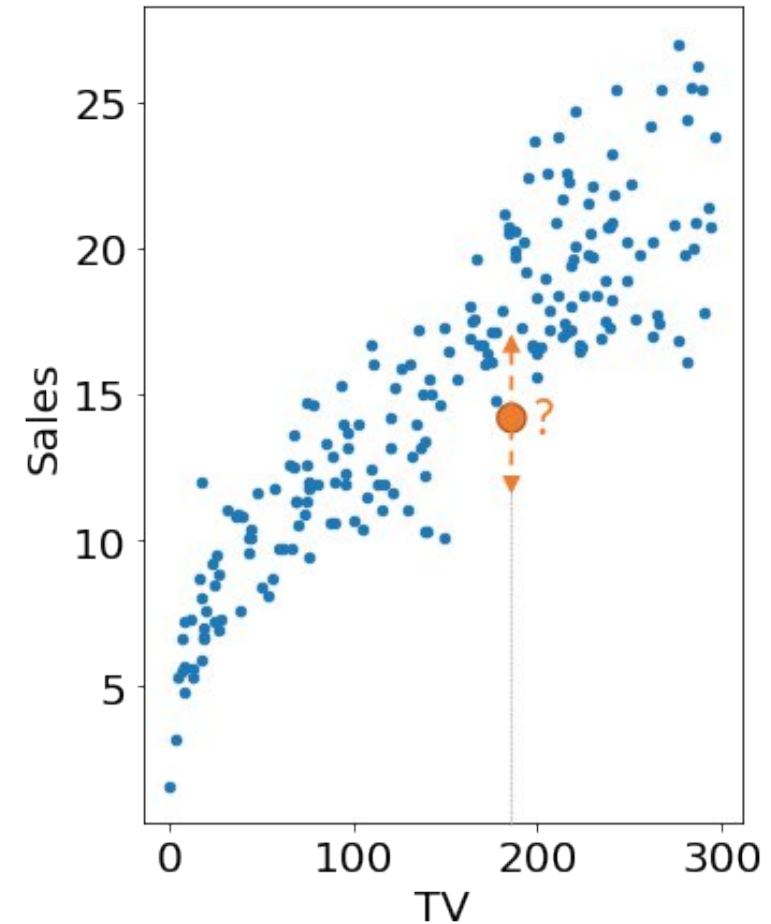
Aufgabe: Vorhersage

Trainingsdaten: bekannte TV-Budgets x_i und Umsätze y_i , $i = 1, \dots, 200$

Gesucht: Vorhersage $\hat{y}$ für einen **neuen** Wert x

Notation:

- x : TV-Budget [T€] (Feature / Prädiktor)
- y : Umsatz [M€] (Zielvariable / Label)
- $\hat{y}$: Vorhersage (Modelloutput)



Mathematische Modellierung

Wir nehmen an:

$$y = f(x) + \epsilon$$

$f: \mathbb{R} \rightarrow \mathbb{R}$: unbekannte Funktion

ϵ : zufälliger Fehlerterm mit Erwartungswert 0

Ansatz: f ist **linear**:

$$f(x) = w_0 + w_1 x$$

Ziel: Parameter w_0, w_1 so wählen, dass f die Trainingsdaten gut beschreibt **und** auf neuen Daten gut vorhersagt.

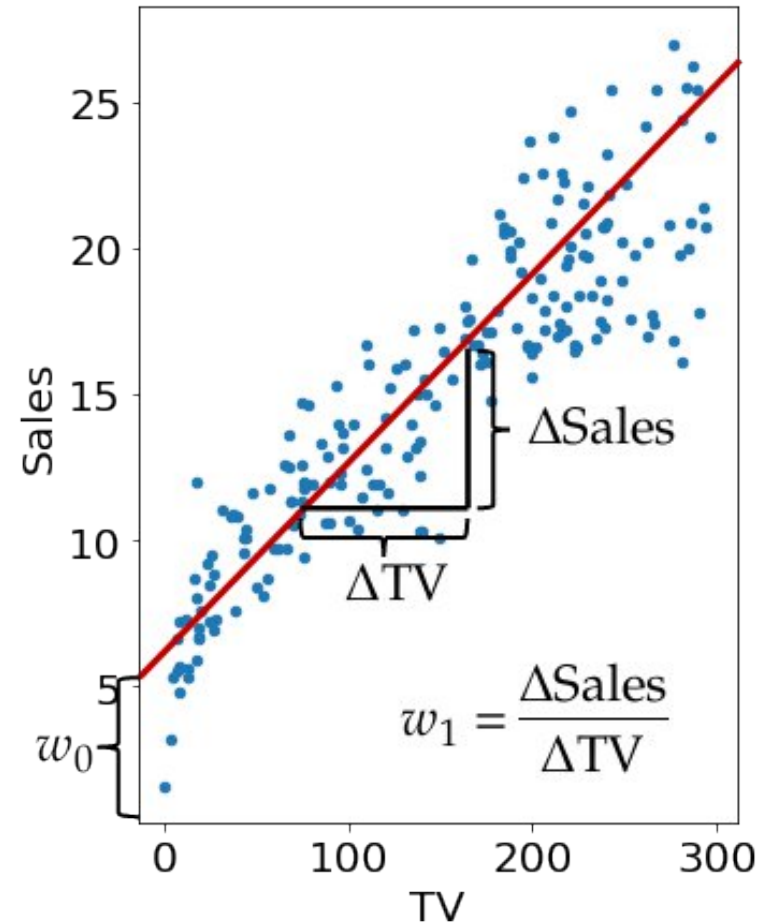
Die Modellfunktion

Gerade $f(x) = w_0 + w_1x$:

- w_0 : Achsenabschnitt (Umsatz bei TV-Budget = 0)
- w_1 : Steigung (Umsatzzuwachs pro T€ TV-Budget)

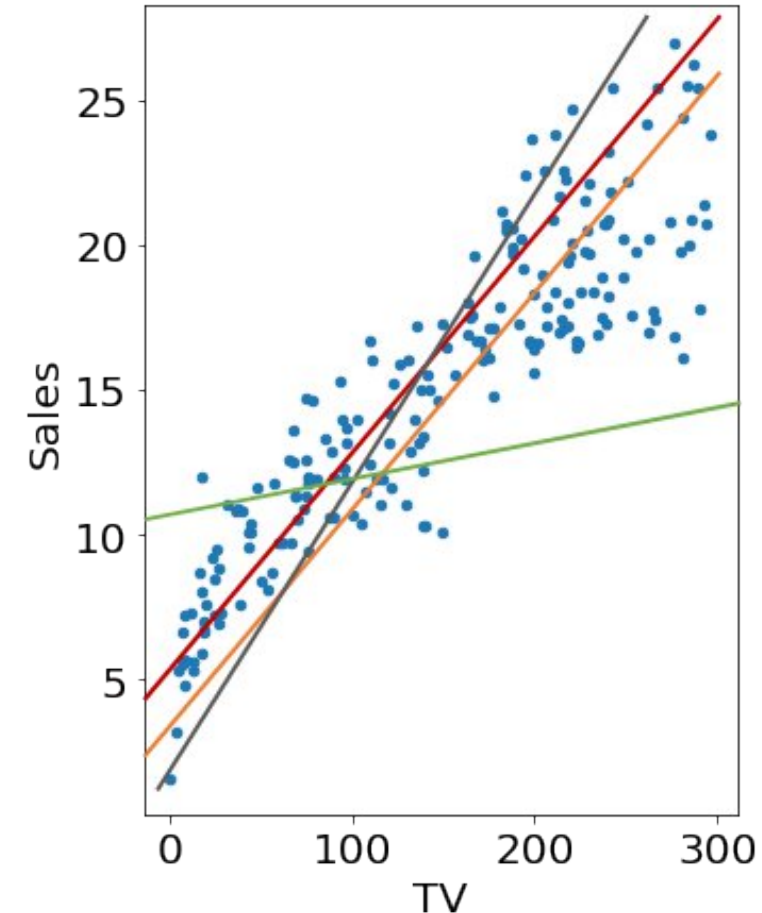
w_0, w_1 : **Modellparameter** (Gewichte)

Wir nennen f auch **Modellfunktion** oder **Modell**.



Welche Gerade ist die beste?

- Es gibt unendlich viele Geraden — welche ist die **beste**?
- Wir brauchen ein Maß dafür, wie gut eine Gerade die Daten beschreibt!



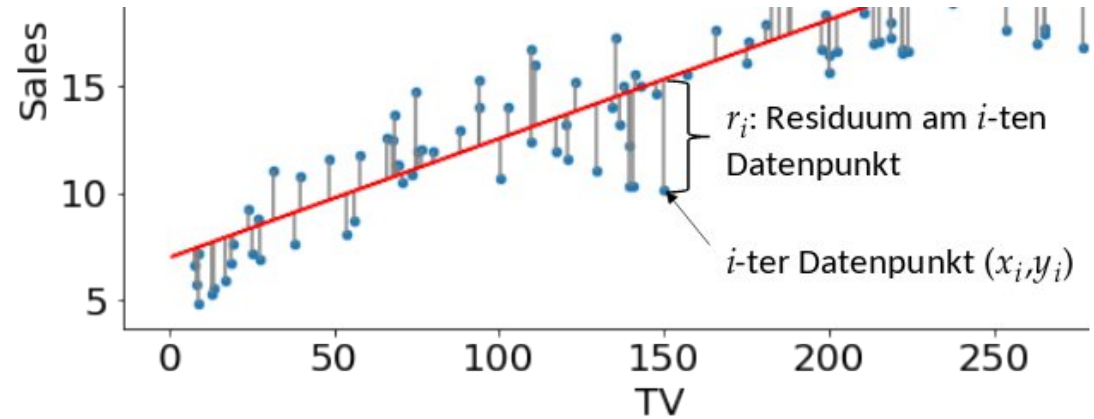
Vier Geraden für vier verschiedene Paare (w_0, w_1)

Residuen



Residuum des i -ten Datenpunkts:

$$r_i = y_i - f(x_i) = \underbrace{y_i}_{\text{Echter Wert}} - \underbrace{(w_0 + w_1 x_i)}_{\substack{\text{Vorhersage der Modellfunktion} \\ (= \text{Punkt auf der Geraden})}}$$



Residuen (graue Streifen) für ein mögliches Modell

$r_i > 0$: Modell unterschätzt den Umsatz

$r_i < 0$: Modell überschätzt den Umsatz

Ziel: Residuen sollen **insgesamt** klein sein

Mean Squared Error (MSE)

Da r_i positiv oder negativ sein kann, betrachten wir das **Quadrat** der Residuen.

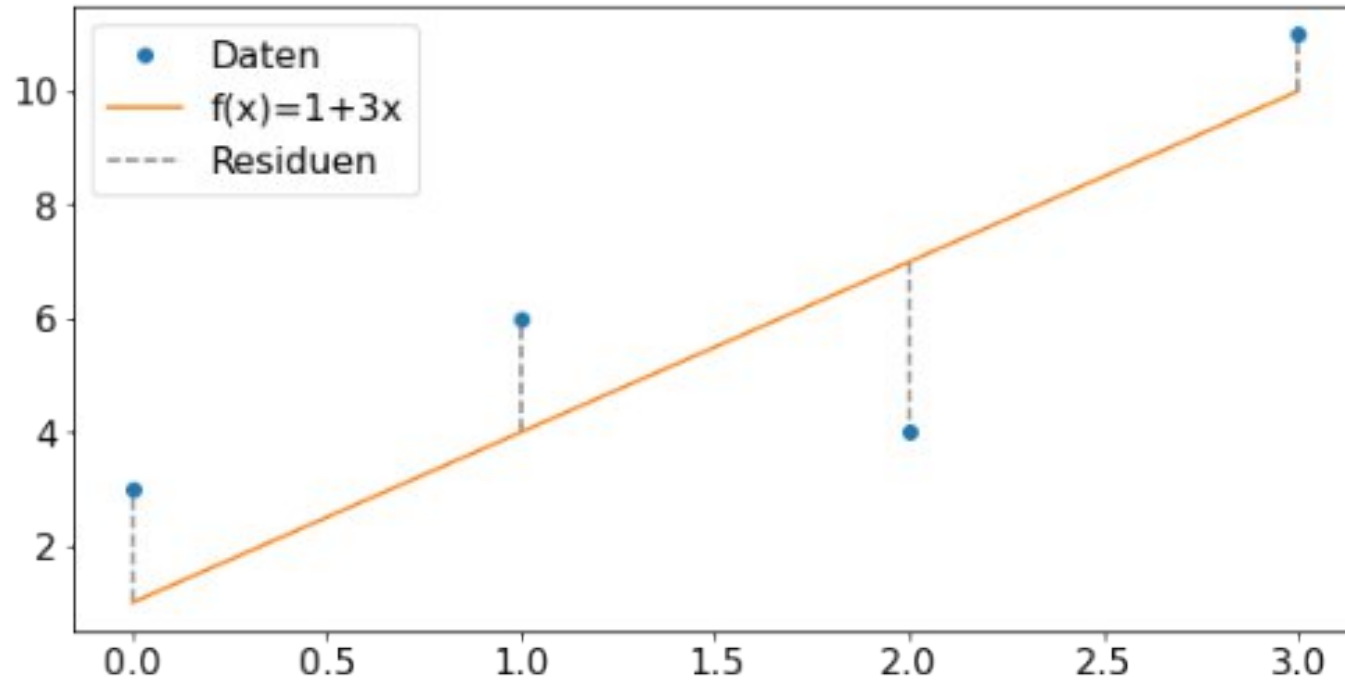
Mittlerer quadratischer Fehler (MSE):

$$MSE = \frac{1}{N} \sum_{i=1}^N r_i^2 = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 x_i))^2$$

N : Anzahl der Trainingsdaten (hier: $N = 200$)

Der MSE hängt von den Parametern w_0 und w_1 ab, für jede Gerade erhalten wir einen anderen MSE

Beispiel: MSE berechnen



x	y
0	3
1	6
2	4
3	11

Die Loss-Funktion

Wir machen die Abhängigkeit von den Parametern explizit:

$$L(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 x_i))^2$$

$L: \mathbb{R}^2 \rightarrow \mathbb{R}$ ist die **Verlustfunktion (Loss-Funktion)**

Der Funktionswert $L(w_0, w_1)$ heißt **Loss** (Verlust)

Modelltraining = Finden von $\hat{w}_0, \hat{w}_1$, die L minimieren:

$$(\hat{w}_0, \hat{w}_1) = \arg \min_{w_0, w_1} L(w_0, w_1)$$

Modeltraining = Loss-Minimierung

Das Minimieren von L ist ein Problem der **mathematischen Optimierung**

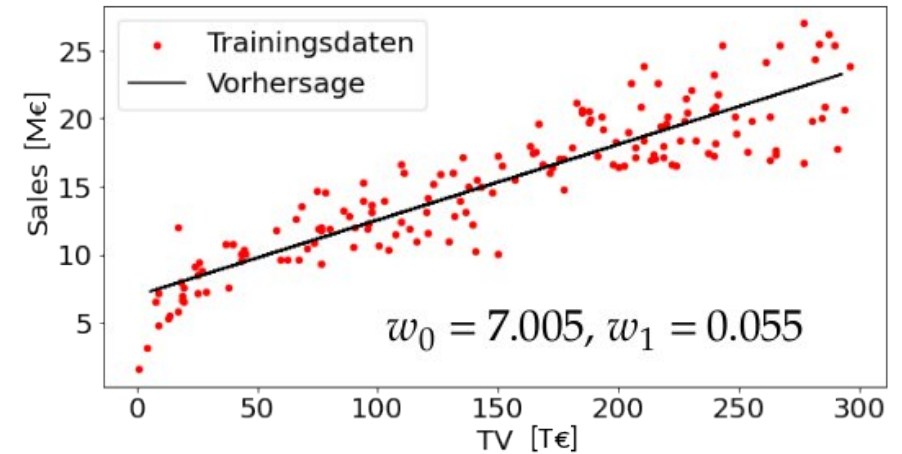
Details im Kapitel „Optimierung“

Lösung der linearen Regression



Für den Advertising-Datensatz:

$$\hat{w}_0 = 7.005, \quad \hat{w}_1 = 0.055$$



Lineare Regression auf dem
Advertising-Datensatz

Train-Test-Split

Modellgüte darf **nicht** auf den Trainingsdaten bewertet werden.

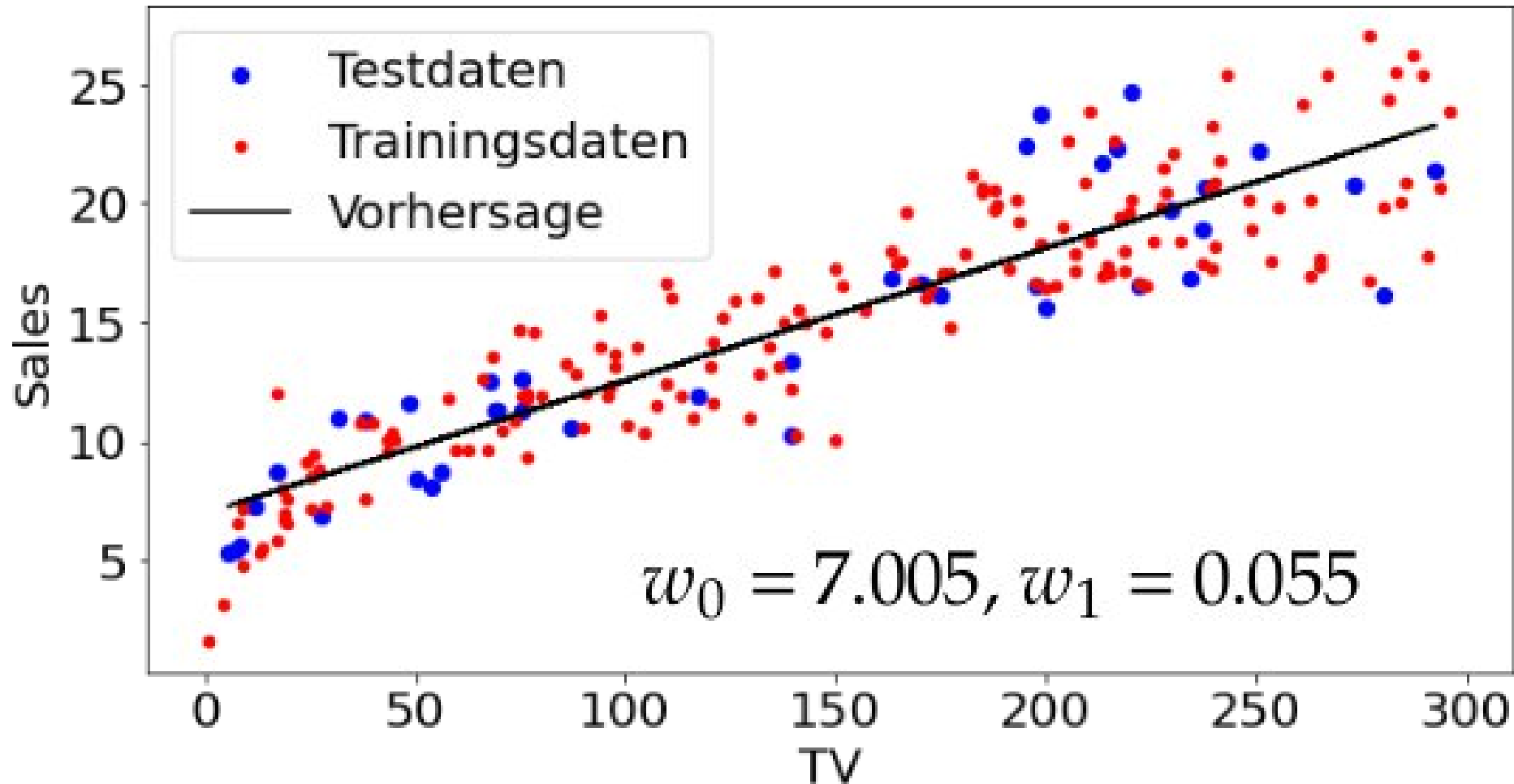
Typisches Vorgehen:

- **Trainingsdaten** (z. B. 80 %): Modell wird darauf trainiert
- **Testdaten** (z. B. 20 %): Modell wird darauf evaluiert

Testdaten bleiben für das Modell während des Trainings unsichtbar



Ergebnisse auf Trainings- und Testdaten



Advertising-Datensatz mit Train-Test-Split



RMSE und MAE

MSE hat Einheit $[\text{M€}]^2 \rightarrow$ schwer interpretierbar.

Root Mean Squared Error (RMSE):

$$RMSE = \sqrt{MSE} \approx \sqrt{5} \approx 2.2 \text{ M€}$$

Interpretation: Modell liegt typischerweise um **2.2 M€** daneben.

Mean Absolute Error (MAE): robust gegenüber Ausreißern:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - f(x_i)|$$

R^2 -Wert (Bestimmtheitsmaß)



Maß unabhängig von der Einheit der Zielvariablen:

$$R^2 = 1 - \frac{RSS}{TSS},$$

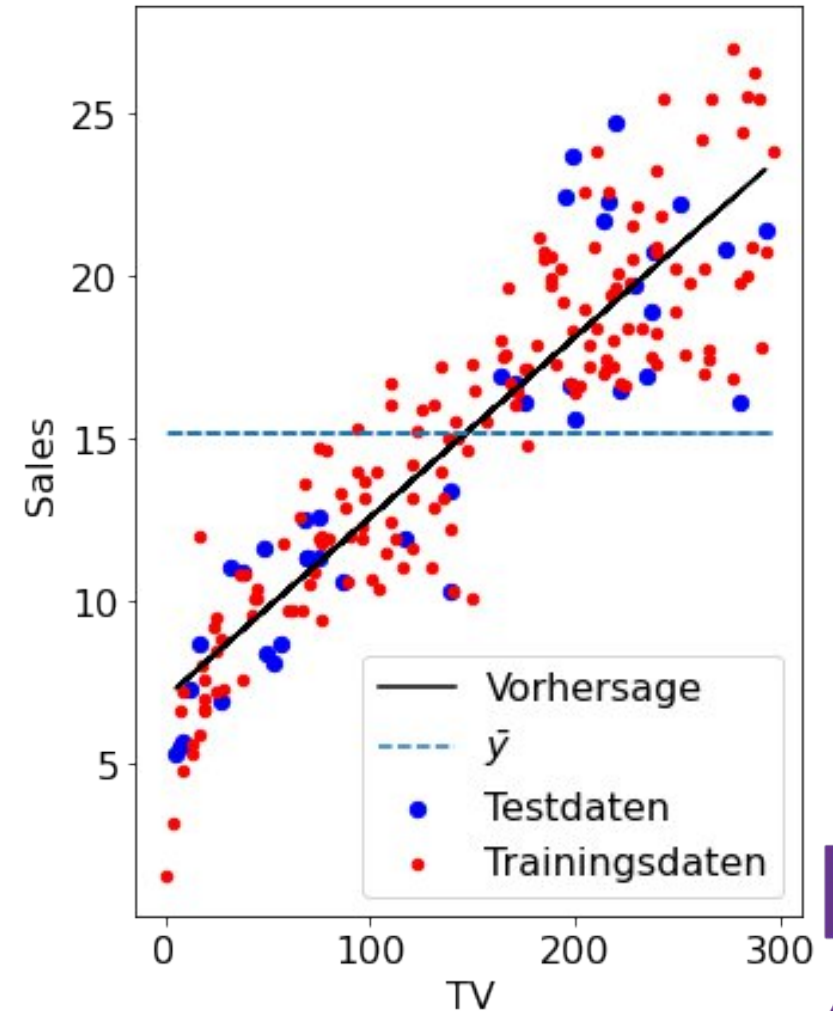
$$RSS = \sum (y_i - f(x_i))^2, \quad TSS = \sum (y_i - \bar{y})^2$$

R^2 = Anteil der **Varianz** in y , der durch das Modell erklärt wird

$R^2 = 1$: Modell perfekt

$R^2 = 0$: Modell so gut wie der Mittelwert $\bar{y}$

Für das Advertising-Beispiel: $R^2 \approx 0.8$



Zusammenfassung bisher

Was haben wir bis jetzt gemacht?

Gegeben: Trainingsdatensatz aus Paaren (x_i, y_i) , $i = 1, \dots, N$.

Ziel: Trainiere Vorhersagemodell, welches für gegebenes x_i möglichst genau y_i vorhersagt.

Vorgehen:

Unterteile Datensatz in Trainings- und Testdaten.

Trainiere lineare Regression auf Trainingsdaten, d.h. finde Minimum der Loss-Funktion.

Ermittle Vorhersagegenauigkeit des Modells auf ungesehenen Daten:

Nutze trainiertes Modell (d.h. die Modellfunktion f) für Vorhersagen auf Testdaten und ermittle RMSE und R^2 .

Zusammenfassung bisher

Dieses Vorgehen ist (nahezu) universell im Überwachten Lernen:

Gegeben: Trainingsdatensatz aus Paaren (x_i, y_i) , $i = 1, \dots, N$.

Ziel: Trainiere Vorhersagemodell, welches für gegebenes x_i möglichst genau y_i vorhersagt.

Vorgehen:

Unterteile Datensatz in Trainings- und Testdaten.

Trainiere **<SupervisedLearningModel>** auf Trainingsdaten, d.h. finde Minimum der Loss-Funktion.

Ermittle Vorhersagegenauigkeit des Modells auf ungesehenen Daten:

Nutze trainiertes Modell (d.h. die Modellfunktion f) für Vorhersagen auf Testdaten und ermittle **<Metric>**.



Multiple linear Regression



Motivation: Mehr Features nutzen

Bisher: nur ein Feature (TV-Budget)

Typischerweise hat ein ML-Modell **viele Features**

Advertising-Datensatz: TV, Radio, Zeitung

Können wir die Vorhersage durch weitere Features verbessern?

	Feature			Label
	TV [T€]	Radio [T€]	Newspaper [T€]	Sales [M€]
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9
...	...	...	...	...
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	14.0
197	177.0	9.3	6.4	14.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	18.4

Datensatz mit einem Feature
(TV)

Zwei Features: TV und Radio

Modellfunktion mit **zwei Features**

($x_1 = \text{TV}$, $x_2 = \text{Radio}$):

$$f(x_1, x_2) = w_0 + w_1 x_1 + w_2 x_2$$

w_j : Umsatzveränderung pro Einheit Feature j bei gleichbleibendem anderen Feature

	Features		Label	
	TV [T€]	Radio [T€]	Newspaper [T€]	Sales [M€]
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9
...	...	...	...	...
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	14.0
197	177.0	9.3	6.4	14.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	18.4

Datensatz mit zwei Features

Multiple lineare Regression (Zwei Features)



Die Modellfunktion

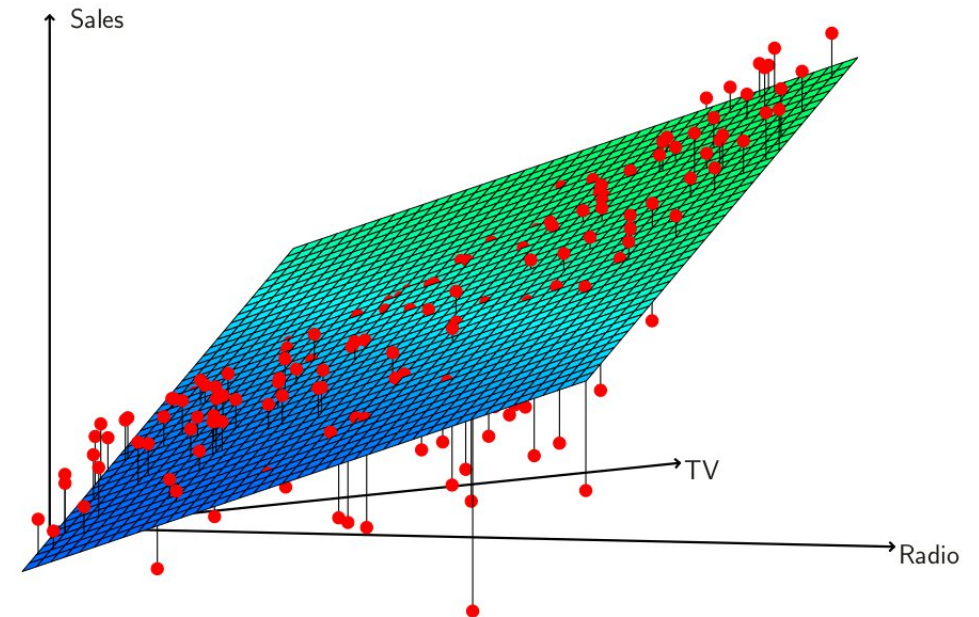
$$f(x_1, x_2) = w_0 + w_1 x_1 + w_2 x_2$$

weist jedem Feature-Paar (x_1, x_2) einen Schätzwert $\hat{y} = f(x_1, x_2)$ zu.

Die Modellfunktion $f(x_1, x_2)$ für zwei Features definiert eine **Ebene** im drei-dimensionalen Raum.

Man erhält die Lösung $\hat{w}_0, \hat{w}_1, \hat{w}_2$ durch Minimieren der **Loss-Funktion**:

$$L(w_0, w_1, w_2) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 x_{i1} + w_2 x_{i2}))^2$$



Ergebnis: Trainierte Ebene

Ergebnis des Modelltrainings:

$$\hat{w}_0 = 4.819, \hat{w}_1 = 0.054, \hat{w}_2 = 0.102$$

Interpretation:

Pro T€ TV-Budget (bei gleichem Radio-Budget): + 54 T€ Umsatz

Pro T€ Radio-Budget (bei gleichem TV-Budget): + 102 T€ Umsatz

	Feature: TV	Features: TV + Radio
RMSE (Train / Test)	2.47 / 2.24	1.68 / 1.63
R^2 (Train / Test)	0.81 / 0.80	0.90 / 0.91

Beliebig viele Features

Allgemeine Form mit p Features:

$$f(x_1, x_2, \dots, x_p) = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_p x_p$$

Verlustfunktion:

$$L(w_0, \dots, w_p) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 x_{i1} + \dots + w_p x_{ip}))^2$$

Doppelindex: x_{ij} = Wert von Feature j für Datenpunkt i .

Einschub: Zur Notation in dieser Vorlesung

Lateinische Kleinbuchstaben wie x, y, a usw. bezeichnen (meist reelle) Zahlen

Fettgedruckte lateinische Kleinbuchstaben wie $\mathbf{x}, \mathbf{y}, \mathbf{a}$ usw. bezeichnen Vektoren im $\mathbb{R}^n$. Deren Komponenten sind reelle Zahlen und werden mit einem (Subscript-)Index indiziert, z.B.:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{pmatrix}$$

Fettgedruckte lateinische Großbuchstaben wie $\mathbf{A}, \mathbf{X}, \mathbf{W}$ usw. bezeichnen $m \times n$ -Matrizen. Deren Einträge (reelle Zahlen) sind die zugehörigen Kleinbuchstaben und werden mit zwei Subscript-Indizes indiziert:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & \cdots & \cdots & \vdots \\ \vdots & & & \vdots \\ a_{m1} & \cdots & \cdots & a_{mn} \end{pmatrix}$$

Vektorschreibweise

Features und Parameter als **Spaltenvektoren**:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{pmatrix} \in \mathbb{R}^p, \quad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \end{pmatrix} \in \mathbb{R}^p$$

Modellfunktion und Verlustfunktion kompakt:

$$f(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x}, \quad L(w_0, \mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + \mathbf{w}^T \mathbf{x}_i))^2$$

Matrixschreibweise: Feature-Matrix

Alle Trainingsdaten in einer **Feature-Matrix** $\mathbf{X} \in \mathbb{R}^{N \times (p+1)}$:

$$\mathbf{X} = \begin{pmatrix} x_{1,1} & \cdots & x_{1,j} & \cdots & x_{1,p} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{i,1} & \cdots & x_{i,j} & \cdots & x_{i,p} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{N,1} & \cdots & x_{N,j} & \cdots & x_{N,p} \end{pmatrix}$$

1 Zeile = 1 Datenpunkt

1 Spalte = 1 Feature (des gesamten Datensatzes)

Labels: Vektor $\mathbf{y} = (y_1, \dots, y_N)^T \in \mathbb{R}^N$

Parameter: Vektor $\mathbf{w} = (w_0, w_1, \dots, w_p)^T \in \mathbb{R}^{p+1}$

Loss-Funktion in Matrixschreibweise

Modellvorhersagen als Matrix-Vektor Produkt:

$$\mathbf{X}\mathbf{w} = \begin{pmatrix} x_{1,1} & \cdots & x_{1,p} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \cdots & x_{N,p} \end{pmatrix} \begin{pmatrix} w_1 \\ \vdots \\ w_p \end{pmatrix} = \begin{pmatrix} w_1 x_{1,1} + \dots + w_p x_{1,p} \\ \vdots \\ w_1 x_{N,1} + \dots + w_p x_{N,p} \end{pmatrix}$$

Damit können wir die Residuen r_i ebenfalls als Vektor schreiben:

$$\mathbf{r} = \mathbf{y} - \mathbf{X}\mathbf{w} = \begin{pmatrix} y_1 - (w_1 x_{1,1} + \dots + w_p x_{1,p}) \\ \vdots \\ y_N - (w_1 x_{N,1} + \dots + w_p x_{N,p}) \end{pmatrix} \in \mathbb{R}^N$$

Die Summe der Residuenquadrate kann man dann schreiben als:

$$\sum_{i=1}^N r_i^2 = \mathbf{r}^T \mathbf{r} = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

Konstanter Term w_0

Mit folgendem Trick kann man auch den sog. *Intercept* w_0 in der Matrix $\mathbf{X}$ unterbringen:
Man erstellt ein zusätzliches „Feature“, das für jeden Datenpunkt den Wert 1 hat:

$$\mathbf{X}\mathbf{w} = \begin{pmatrix} \color{red}{1} & x_{1,1} & \cdots & x_{1,p} \\ \vdots & \vdots & \ddots & \vdots \\ \color{red}{1} & x_{N,1} & \cdots & x_{N,p} \end{pmatrix} \begin{pmatrix} \color{red}{w_0} \\ w_1 \\ \vdots \\ w_p \end{pmatrix} = \begin{pmatrix} \color{red}{w_0} + w_1 x_{1,1} + \dots + w_p x_{1,p} \\ \vdots \\ \color{red}{w_0} + w_1 x_{N,1} + \dots + w_p x_{N,p} \end{pmatrix}$$

—> Wenn wir den Ausdruck $\mathbf{X}\mathbf{w}$ verwenden, so ist damit normalerweise der Fall inkl. Intercept (w_0) inbegriffen (dadurch muss man w_0 nicht immer gesondert aufführen).

Lineares Regressionsproblem

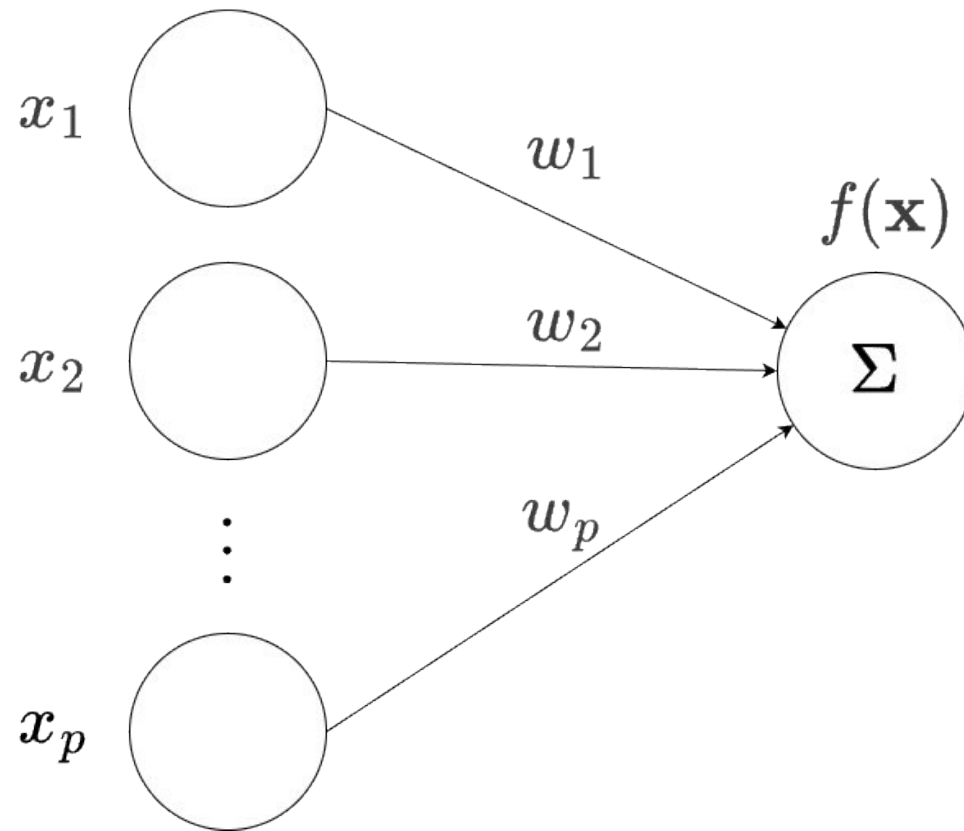
Zusammenfassung:

Lineare Regression wird trainiert, indem folgendes **Minimierungsproblem** gelöst wird:

$$\min_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \left(y_i - (w_1 x_{i,1} + \dots w_p x_{i,p}) \right)^2 = \frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

—> Wie? Kommt im Optimierungskapitel.

Alternative Darstellung: als neuronales Netz



Eingabeknoten = Features

Kante = Parameter mit Gewicht w_j

Ausgabeknoten = gewichtete Summe der Eingaben



Nichtlineare Features



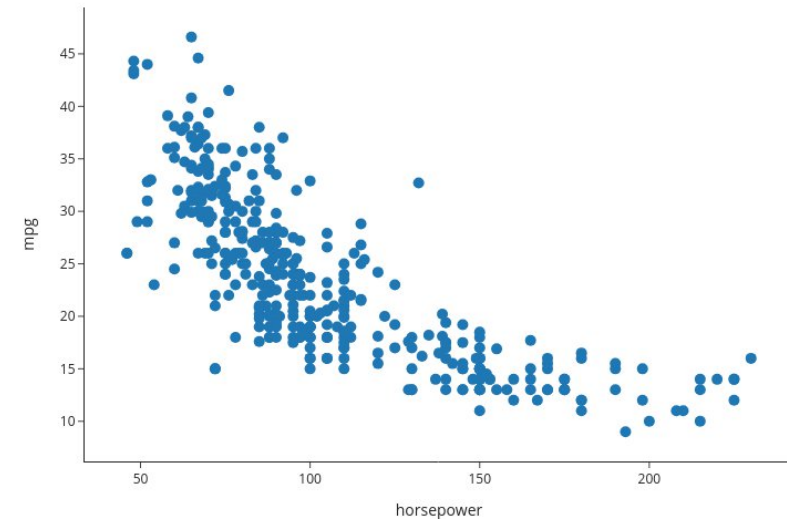
Motivation: Nichtlineare Zusammenhänge

Das lineare Modell setzt einen **linearen** Zusammenhang voraus

Das ist oft nicht gerechtfertigt

Beispiel: auto-Datensatz

- Feature: horsepower (PS)
- Zielvariable: mpg (Meilen pro Gallone — hoher Wert = niedriger Verbrauch)



Auto-Datensatz: PS
vs. Verbrauch

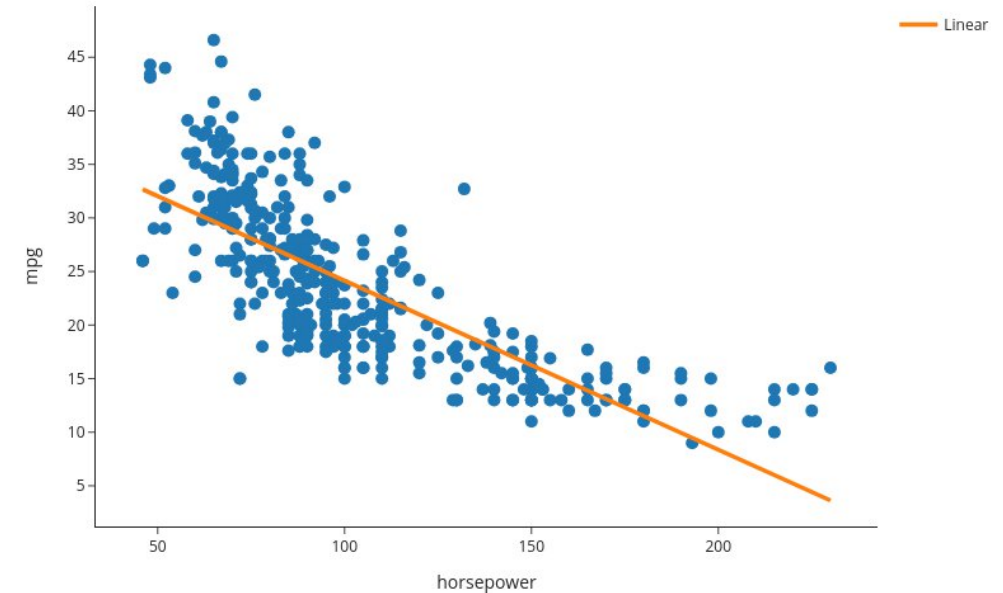
Lineares Modell auf dem Auto-Datensatz

Lineares Modell: $R^2 = 0.605$

Beschreibt die Form der Daten **nicht gut**

Bei niedrigen und hohen PS-Werten: Verbrauch wird unterschätzt

Hinweis auf **nichtlinearen** Zusammenhang



Lineares Modell ($R^2 = 0.605$)

Polynomielle Regression

Idee: Modell als **quadratische** Funktion in horsepower:

$$\text{mpg} = w_0 + w_1 \cdot hp + w_2 \cdot hp^2 + \epsilon$$

Trick: **zwei Features** verwenden:

$$x_1 = hp, \quad x_2 = hp^2$$

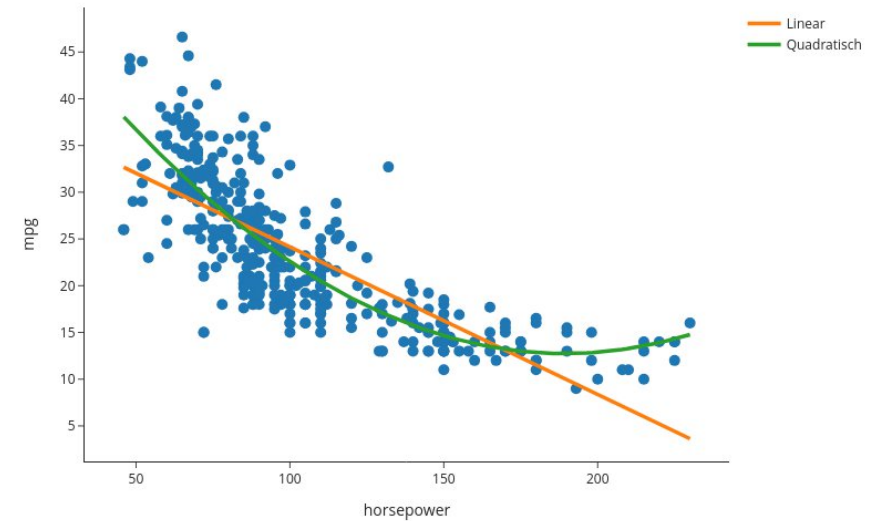
f ist nichtlinear in hp , aber **linear in $\mathbf{w}$** — kompatibel mit unserem Framework!

Quadratisches Modell

Quadratisches Modell: $R^2 = 0.688$

Beschreibt die Form der Daten deutlich besser

R^2 verbessert sich von 0.605 auf 0.688



Quadratisches Modell ($R^2 = 0.688$)

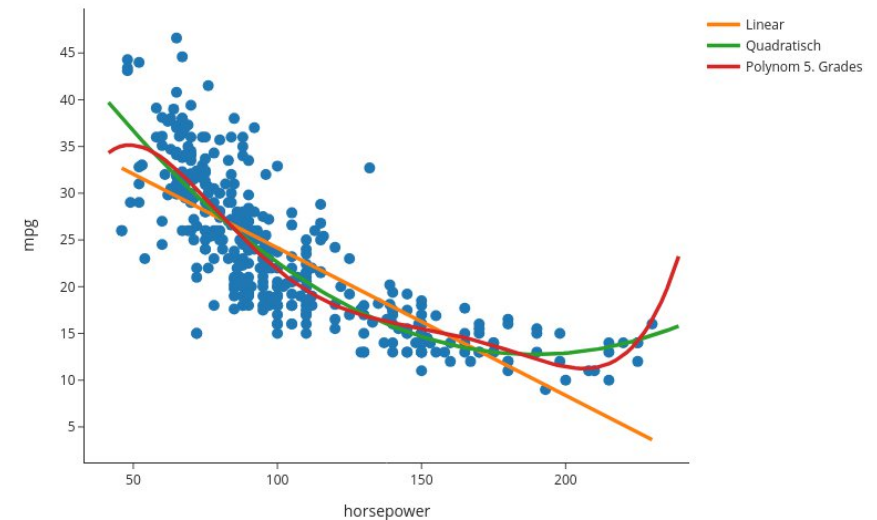
Polynom 5. Grades

Polynom 5. Grades: $R^2 = 0.697$

Nur minimale Verbesserung gegenüber dem quadratischen Modell

Die Kurve wirkt unnötig „verwackelt“

Warnung: Zu viele Features können zu **Overfitting** führen!



Polynom 5. Grades ($R^2 = 0.697$)

Allgemeine nichtlineare Features

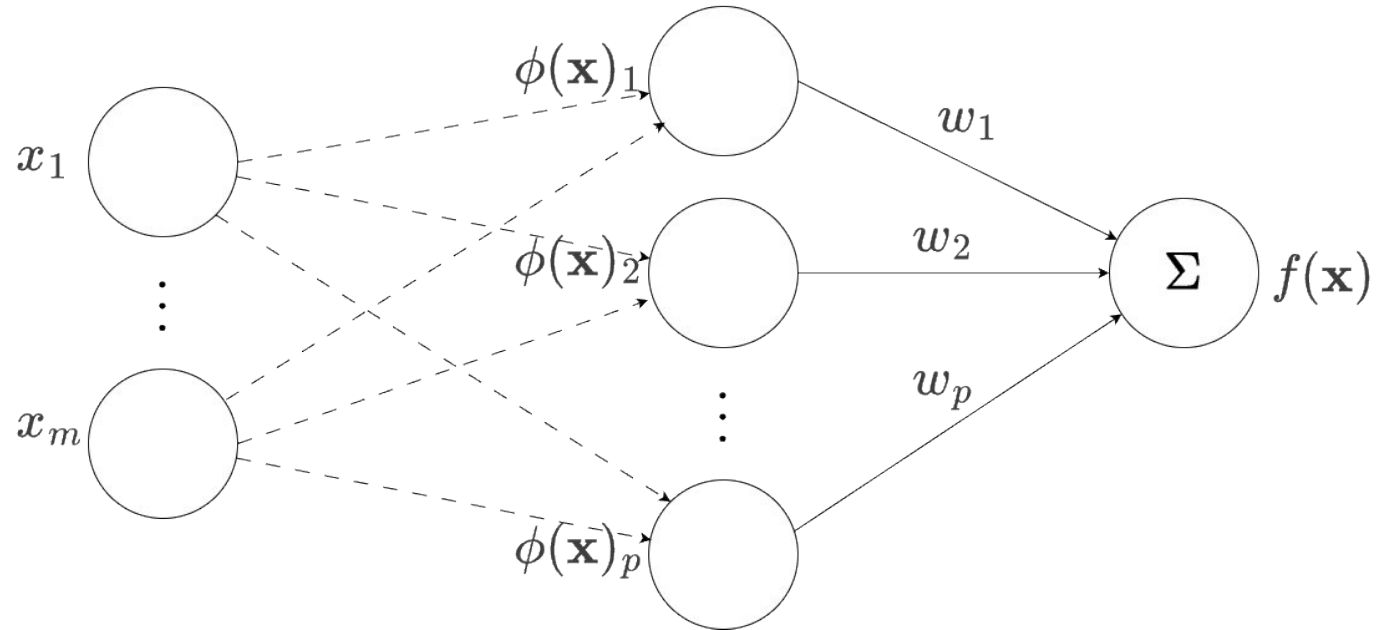
Feature-Engineering als Funktion ϕ ,
die Rohdaten transformiert:

$$\phi: \mathbb{R}^m \rightarrow \mathbb{R}^p$$

Modellfunktion auf den
transformierten Features:

$$f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x})$$

ϕ kann beliebig komplex sein
(Polynome, sin, exp, Domänenwissen)



Das Modell f bleibt **linear in $\mathbf{w}$**

Feature-Engineering als vorgeschaltete
Transformation



Bias-Variance-Tradeoff



Beispiel: Polynominterpolation

—> **Skript 2.4**

Zufällige Daten aus einer kubischen Funktion + Rauschen

Trainings-Test-Split: 11 Trainingspunkte, 9 Testpunkte

Polynome vom Grad 0 bis 10 werden trainiert

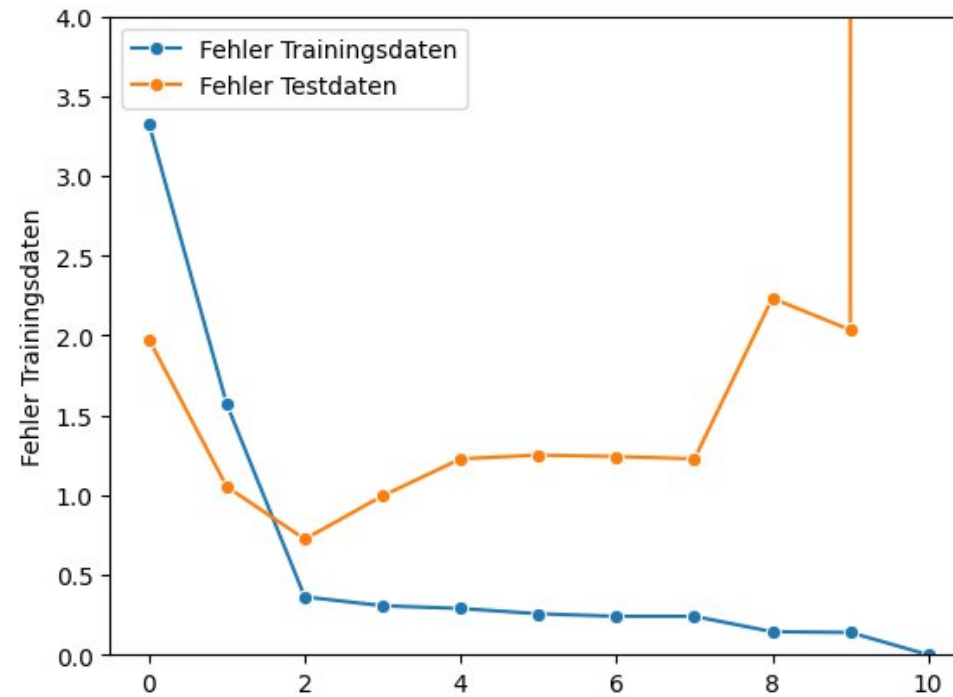
Beobachtung:

Grad 0–1: schlechte Anpassung (zu einfach)

Grad 2–3: gute Anpassung

Grad 9–10: perfekte Anpassung an Trainingsdaten, aber sehr schlechte
Generalisierung

Fehler auf Trainings- vs. Testdaten



1. Fehler auf Trainingsdaten wird **immer kleiner** mit steigendem Polynomgrad
 2. Fehler auf Testdaten wird zunächst kleiner, steigt dann wieder an
- Es gibt einen **optimalen** Polynomgrad.

Underfitting und Overfitting

Underfitting

Modell zu einfach

Fehler auf Trainings- **und** Testdaten groß

Auch: hoher **Bias**

Overfitting

Modell zu komplex

Fehler auf Trainingsdaten klein, auf
Testdaten groß

„Modell lernt Trainingsdaten auswendig“

Auch: hohe **Variance**

Bias-Variance-Tradeoff

Bias: Verzerrung durch zu starke (falsche) Modellannahmen

Beispiel: lineares Modell für nichtlinearen Zusammenhang

Variance: Variabilität des gelernten Modells bei unterschiedlichen Trainingsdaten

Polynom hohen Grades: sehr unterschiedliche Kurven je nach Trainings-Set

Modellkomplexität	Bias	Variance
niedrig	hoch	niedrig
hoch	niedrig	hoch



Ziel: **Generalisierung** — Modell, das auf neuen, unbekannten Daten gut funktioniert.

—> Mehr dazu im Kapitel „Generalisierung“



Zusammenfassung

Konzept	Bedeutung
Modellfunktion	$f(\mathbf{x}) = w_0 + \mathbf{w}^T \mathbf{x}$
Verlustfunktion	$L(\mathbf{w}) = \frac{1}{N} \ \mathbf{y} - \mathbf{X}\mathbf{w}\ _2^2$ (MSE)
Training	Minimierung des Loss → Kapitel 3
Evaluierung	RMSE, MAE, R^2 auf Testdaten
Erweiterungen	Nichtlineare Features
Bias-Variance	Trade-Off zwischen Unter- und Überanpassung